Universidade de Brasília

Departamento de Ciência da Computação

Relatório do Trabalho 1 de Programação Concorrente

Evelyn Soares Pereira 17/0102785

Brasília
10 de novembro de 2026

1 Introdução

Em programação concorrente, *threads* que cooperam por memória compartilhada correm o risco de *condições de corrida*: se duas delas acessam o mesmo dado sem coordenação e pelo menos uma escreve, o resultado passa a depender da ordem em que o escalonador intercala suas instruções. Como essa ordem muda a cada execução, os erros aparecem em uma execução e podem sumir na seguinte. Para evitá-los, as *threads* são sincronizadas com mecanismos como *locks*, variáveis de condição e semáforos.

Este trabalho aplica esses mecanismos a um problema de Manna e Pnueli, apresentado por Ben-Ari (2006, Apêndice C, exercício 8): calcular o coeficiente binomial $\binom{n}{k}$ com dois processos, um para o numerador e outro para o denominador. O denominador entrega resultados parciais ao numerador, que divide assim que os recebe, impedindo que os valores intermediários cresçam demais. Essa comunicação acontece por memória compartilhada e, sem sincronização, está sujeita a condições de corrida. A solução foi implementada em C com POSIX Pthreads. A Seção 2 formaliza o problema, a Seção 3 descreve o algoritmo, a implementação e os testes, e a Seção 4 conclui o trabalho.

2 Formalização do Problema Proposto

2.1 O Coeficiente Binomial

Para inteiros $0 \le k \le n$,

$$\binom{n}{k} = \frac{n!}{(n-k)!\,k!} = \frac{n \cdot (n-1) \cdots (n-k+1)}{1 \cdot 2 \cdots k}, \tag{1}$$

ou seja, há k fatores no numerador e k no denominador. Calcular o numerador inteiro e só dividir no final é inviável mesmo para valores modestos: em $\binom{30}{15}$, o numerador $30 \cdot 29 \cdots 16 \approx 2{,}0 \times 10^{20}$ não cabe em 64 bits ($\approx 1{,}8 \times 10^{19}$), embora o resultado ($155\,117\,520$) caiba em 32. A proposta do exercício é intercalar as operações: no passo i, multiplicar o valor acumulado pelo i-ésimo fator do numerador e dividir pelo i-ésimo fator do denominador (Tabela 1).

Tabela 1: Cálculo de $\binom{10}{3}$ intercalando multiplicações e divisões.

Passo i	Multiplica por	Antes da divisão	Divide por	Depois da divisão
1	10	10	1	10
2	9	90	2	45
3	8	360	3	120

Como a divisão inteira em C descarta o resto sem aviso, esse método só funciona se toda divisão intermediária for exata, o que é provado a seguir.

2.2 Divisibilidade dos Resultados Parciais

Chamamos de $P(i, j)$ o produto de i inteiros consecutivos começando em j:

$$P(i, j) = j \cdot (j+1) \cdots (j+i-1). \qquad \text{Por exemplo, } P(3, 5) = 5 \cdot 6 \cdot 7 = 210.$$

Lema 1. *Para todos os inteiros $i \ge 1$ e $j \ge 1$, $i!$ divide $P(i, j)$.*

A demonstração usa uma identidade auxiliar. Para $i \geq 2$, os produtos $P(i, j)$ e $P(i, j+1)$ têm em comum os fatores $(j+1) \cdots (j+i-1)$, que formam $P(i-1,\, j+1)$. Assim,

$$P(i, j) = j \cdot P(i-1,\, j+1) \qquad \text{e} \qquad P(i, j+1) = P(i-1,\, j+1) \cdot (j+i),$$

e, subtraindo uma da outra,

$$P(i, j+1) - P(i, j) = \big[(j+i) - j\big] \cdot P(i-1,\, j+1) = i \cdot P(i-1,\, j+1). \tag{2}$$

Por exemplo, $P(3, 6) - P(3, 5) = 6 \cdot 7 \cdot 8 - 5 \cdot 6 \cdot 7 = (8-5) \cdot 6 \cdot 7 = 3 \cdot P(2, 6)$.

Demonstração. Por indução em i.

Base ($i = 1$): $P(1, j) = j$, e $1! = 1$ divide qualquer inteiro.

Passo: seja $i \geq 2$ e suponha, como hipótese de indução, que $(i-1)!$ divide $P(i-1, m)$ para todo $m \geq 1$. Vamos mostrar, agora por indução em j, que $i!$ divide $P(i, j)$ para todo $j \geq 1$.

- $j = 1$: $P(i, 1) = 1 \cdot 2 \cdots i = i!$, que é divisível por $i!$.
- De j para $j+1$: suponha que $i!$ divide $P(i, j)$. Pela identidade (2),

  $$P(i, j+1) = P(i, j) + i \cdot P(i-1,\, j+1).$$

  A primeira parcela é múltipla de $i!$ pela suposição. Na segunda, a hipótese de indução (com $m = j+1$) dá $P(i-1,\, j+1) = c \cdot (i-1)!$ para algum inteiro c, logo $i \cdot P(i-1,\, j+1) = c \cdot i \cdot (i-1)! = c \cdot i!$. A soma de dois múltiplos de $i!$ é múltipla de $i!$. □

Aplicação ao problema. Depois de i multiplicações, o numerador acumulou $N_i = n \cdot (n-1) \cdots (n-i+1)$. Lido de trás para a frente, esse é um produto de i inteiros consecutivos começando em $n-i+1$, ou seja, $N_i = P(i,\, n-i+1)$; por exemplo, $10 \cdot 9 \cdot 8 = 8 \cdot 9 \cdot 10 = P(3, 8)$. Como $i \leq k \leq n$, o início $n-i+1$ é pelo menos 1, e o lema garante que $i!$ divide N_i.

Isso explica a Tabela 1. Antes da i-ésima divisão, o acumulado já foi dividido por $1, 2, \ldots, i-1$, ou seja, por $(i-1)!$. Então

$$\underbrace{\frac{N_i}{(i-1)!}}_{\text{antes da divisão}} \div i \;=\; \frac{N_i}{i!} \;=\; \binom{n}{i},$$

que é inteiro pelo lema. No passo 3 da tabela, por exemplo, $720/2! = 360$ e $360/3 = 120 = 720/3!$. Por isso o denominador não precisa enviar $i!$: basta enviar o próximo fator i, e as divisões sucessivas por $1, 2, \ldots, i$ têm o mesmo efeito que dividir por $i!$. Em $\binom{30}{15}$, o maior valor acumulado fica em torno de $2{,}3 \times 10^9$, contra $2{,}0 \times 10^{20}$ do numerador completo.

2.3 Processos, Dados Compartilhados e Condições de Corrida

O programa tem três *threads*:

- **Denominador**: gera, em ordem, os fatores $1, 2, \ldots, k$ e os escreve num *buffer* compartilhado;

- **Numerador**: para $i = 1, \ldots, k$, multiplica o seu acumulado por $n - i + 1$, lê o próximo fator do *buffer* e divide o acumulado por ele;
- **Principal**: cria as outras duas, espera que terminem (`pthread_join`) e imprime o resultado.

O acumulado pertence só ao Numerador (a Principal só o lê depois do `pthread_join`), então não há disputa por ele. O que é compartilhado é o *buffer*: as posições de memória onde o Denominador escreve os fatores e de onde o Numerador os lê, junto com as variáveis que indicam quantos fatores estão disponíveis. Sem sincronização, ele está sujeito a três condições de corrida:

1. **Leitura antes da escrita:** o Numerador lê uma posição em que o Denominador ainda não escreveu e obtém um valor antigo ou o valor inicial (se for 0, divide por zero).
2. **Sobrescrita:** o Denominador escreve um fator por cima de outro que o Numerador ainda não leu, e esse divisor se perde.
3. **Atualização perdida:** se as duas *threads* alteram a mesma variável de controle (por exemplo, `qtd++` no Denominador e `qtd--` no Numerador), cada operação é feita em três passos (ler, alterar, escrever), e uma alteração pode apagar a outra.

A Tabela 2 mostra uma execução de $\binom{5}{2} = 10$ sem sincronização, com um *buffer* de uma posição, em que as corridas 1 e 2 acontecem e o programa imprime 4 sem nenhum erro aparente.

Tabela 2: Execução sem sincronização de $\binom{5}{2}$, passo a passo.

Passo	Denominador	*buffer*	Numerador	Acumulado	Problema
1	–	–	multiplica por 5	5	
2	escreve 1	1	–	5	
3	escreve 2	2	–	5	o 1 se perdeu (corrida 2)
4	–	2	lê 2 e divide: 5/2	2	divisão truncada
5	–	2	multiplica por 4	8	
6	–	2	lê 2 de novo: 8/2	**4**	valor antigo (corrida 1)

Uma solução correta deve garantir:

- **entrega exata:** cada fator é lido pelo Numerador exatamente uma vez, na ordem em que foi escrito;
- **exclusão mútua** no acesso às variáveis do *buffer*;
- **término sem *deadlock*:** as duas *threads* terminam após k iterações.

Com as duas primeiras, o Numerador repete exatamente a Tabela 1 e, pelo Lema 1, obtém $\binom{n}{k}$. Também é desejável que as esperas sejam por bloqueio, e não por espera ocupada.

3 Descrição do Algoritmo Desenvolvido para Solução do Problema Proposto

3.1 Visão Geral

3.2 Mecanismos de Sincronização Utilizados

A solução usa apenas dois semáforos POSIX (`sem_t`) e um *buffer* circular de N posições:

- `posicoes_livres`, iniciado com N, conta as posições do *buffer* que podem ser escritas. O Denominador faz `sem_wait` nele antes de escrever, e o Numerador faz `sem_post` depois de ler.
- `fatores_prontos`, iniciado com 0, conta os fatores escritos e ainda não lidos. O Numerador faz `sem_wait` nele antes de ler, e o Denominador faz `sem_post` depois de escrever.

O semáforo foi escolhido porque o problema é, no fundo, contar recursos: um semáforo é um contador cujas operações são atômicas e que bloqueia a *thread* quando chega a zero, sem espera ocupada. Assim, o `sem_wait` em `fatores_prontos` impede a corrida 1, o `sem_wait` em `posicoes_livres` impede a corrida 2, e a corrida 3 deixa de existir, porque o contador de fatores disponíveis passa a ser o próprio semáforo. O `sem_wait` e o `sem_post` de cada semáforo ficam em *threads* diferentes: aqui os semáforos sinalizam eventos de uma *thread* para a outra, e não fazem exclusão mútua.

Por que não há *mutex*. Com um único escritor e um único leitor, nenhuma variável é alterada pelas duas *threads* ao mesmo tempo. O índice de escrita do *buffer* só é usado pelo Denominador, e o de leitura, só pelo Numerador; n, k e N são constantes; e o resultado só é lido pela Principal após o `pthread_join`. Resta o vetor do *buffer*, e os dois semáforos garantem que o Numerador só lê uma posição depois que o Denominador a escreveu, e que o Denominador só a reescreve depois que o Numerador a leu. Um *mutex* só seria necessário com mais de um escritor ou mais de um leitor, que passariam a disputar o mesmo índice.

3.3 Pseudocódigo

3.4 Corretude

No código, cada `sem_wait` é precedido de um `sem_trywait`, que só serve para imprimir um aviso quando a *thread* vai bloquear; o par tem o mesmo efeito de um único `sem_wait`, e é assim que ele é tratado a seguir.

Sejam L e P os valores de `posicoes_livres` e `fatores_prontos`. Seja $a_d = 1$ enquanto o Denominador já passou do seu `sem_wait` mas ainda não fez o `sem_post` correspondente, e $a_d = 0$ caso contrário; a_n é definido da mesma forma para o Numerador. No início, $L = N$ e $P = a_d = a_n = 0$. O `sem_wait` do Denominador transfere uma unidade de L para a_d, e o seu `sem_post` transfere a_d para P; o Numerador faz o mesmo no sentido contrário, de P para a_n e de a_n para L. Nenhuma operação cria ou destrói unidades, então em todo instante vale o invariante

$$L + P + a_d + a_n = N. \tag{3}$$

Ausência de condições de corrida. O j-ésimo fator é escrito pelo Denominador na posição $(j-1) \bmod N$, e o Numerador lê as posições na mesma ordem circular.

- O Numerador só lê o i-ésimo fator depois do seu i-ésimo `sem_wait` em `fatores_prontos`, que só tem sucesso depois de i `sem_post`s do Denominador, ou seja, depois que o i-ésimo fator foi escrito. Não há leitura antes da escrita (corrida 1).
- O Denominador só escreve o j-ésimo fator depois do seu j-ésimo `sem_wait` em `posicoes_livres`. Como o semáforo começa em N, para $j > N$ isso exige pelo menos $j - N$ `sem_post`s do Numerador, ou seja, que o fator $j - N$, que ocupava a mesma posição, já tenha sido lido. Não há sobrescrita (corrida 2).
- O único contador compartilhado é o valor dos próprios semáforos, e as operações sobre ele são atômicas (corrida 3).

Além disso, no padrão POSIX, `sem_post` e `sem_wait` sincronizam a memória: o valor escrito antes do `sem_post` é o que o Numerador enxerga depois do `sem_wait`. Logo, cada fator é lido exatamente uma vez e na ordem em que foi escrito (entrega exata).

Resultado correto. Com a entrega exata, o Numerador executa exatamente a sequência da Tabela 1: no passo i, multiplica por $n-i+1$ e divide por i. Pela Seção 2.2, cada divisão é exata e o acumulado depois do passo i é $\binom{n}{i}$; ao final, $\binom{n}{k}$.

Ausência de *deadlock*. Uma *thread* só bloqueia em um `sem_wait`. Para haver *deadlock*, as duas teriam de estar bloqueadas ao mesmo tempo: o Denominador em `posicoes_livres`, com $L = 0$ e $a_d = 0$ (*buffer* cheio), e o Numerador em `fatores_prontos`, com $P = 0$ e $a_n = 0$ (*buffer* vazio). Nesse caso $L+P+a_d+a_n = 0 \neq N$, o que contradiz o invariante (3), já que $N \geq 1$. Intuitivamente, o *buffer* não pode estar cheio e vazio ao mesmo tempo.

Término e ausência de *starvation*. Uma *thread* também não fica bloqueada para sempre porque a outra terminou. Se o Numerador terminou, ele fez k `sem_post`s em `posicoes_livres`; se o Denominador espera a sua j-ésima posição ($j \leq k$), ele fez antes $j-1$ `sem_wait`s, então $L = N - (j-1) + k \geq N + 1 > 0$, e ele não estaria bloqueado. Simetricamente, se o Denominador terminou e o Numerador espera o seu i-ésimo fator, $P = k - (i-1) \geq 1$. Assim, se o Denominador está bloqueado ($L = 0$, $a_d = 0$), o invariante dá $P + a_n = N \geq 1$, e o Numerador, que não está bloqueado nem terminou, ou já tem um fator na mão ($a_n = 1$) ou encontrará um pronto ($P \geq 1$); em ambos os casos, ele fará o `sem_post` em `posicoes_livres` em um número finito de passos. O caso do Numerador bloqueado é simétrico. Como só o Denominador espera em `posicoes_livres` e só o Numerador espera em `fatores_prontos`, esse `sem_post` acorda exatamente a *thread* bloqueada: não há várias *threads* disputando o mesmo semáforo e, portanto, não há *starvation*. Como cada laço tem exatamente k iterações, as duas *threads* terminam e os `pthread_join` da Principal retornam. Supõe-se, como de costume, um escalonador justo, em que toda *thread* pronta acaba sendo executada.

3.5 Implementação

A solução está no arquivo `coef_binomial.c`. Para compilar e executar:

```
gcc -Wall -o coef_binomial coef_binomial.c -lpthread
./coef_binomial
```

Os valores de n, k e do tamanho N do *buffer* são constantes definidas no início do código, com padrão $n = 10$, $k = 5$ e $N = 2$ (a execução termina com `Resultado: C(10, 5) = 252`). O *buffer* é menor que k de propósito, para que o Denominador encha o *buffer* e a espera apareça na saída. Para calcular outro coeficiente, basta alterar essas constantes e compilar de novo.

3.6 Testes e Resultados

A Tabela 3 lista casos de teste, com o resultado esperado e o que cada um mostra. Para reproduzir um caso, basta trocar os valores de `VALOR_N`, `VALOR_K` e `TAM_BUFFER` no início do código e compilar de novo. Todos os casos da tabela foram executados e imprimiram o resultado esperado. Por causa das pausas (`sleep`) que simulam o tempo de cada operação, a execução leva cerca de 4 segundos por passo do Numerador, ou seja, cerca de $4k$ segundos.

Tabela 3: Casos de teste sugeridos.

n	k	N	$\binom{n}{k}$ esperado	O que o caso mostra
10	5	2	252	padrão: o *buffer* enche e o Denominador espera
10	3	5	120	exemplo do enunciado (Tabela 1)
5	2	1	10	caso da Tabela 2, agora correto; com $N = 1$, as *threads* se alternam
7	0	2	1	$k = 0$: nenhuma *thread* entra no laço
7	7	2	1	$k = n$
30	15	5	155 117 520	o numerador completo estouraria 64 bits
100	5	5	75 287 520	n grande e k pequeno

A Figura 1 mostra a saída da execução padrão. O Denominador, mais rápido, escreve os fatores 2 e 3, encontra o *buffer* cheio e espera; só volta a escrever depois que o Numerador lê um fator e libera uma posição. Os índices de escrita e leitura alternam entre as posições 0 e 1 do *buffer* circular. Depois de cada divisão, o acumulado vale 10, 45, 120, 210 e 252, ou seja, $\binom{10}{i}$ para $i = 1, \ldots, 5$, como previsto na Seção 2.2.

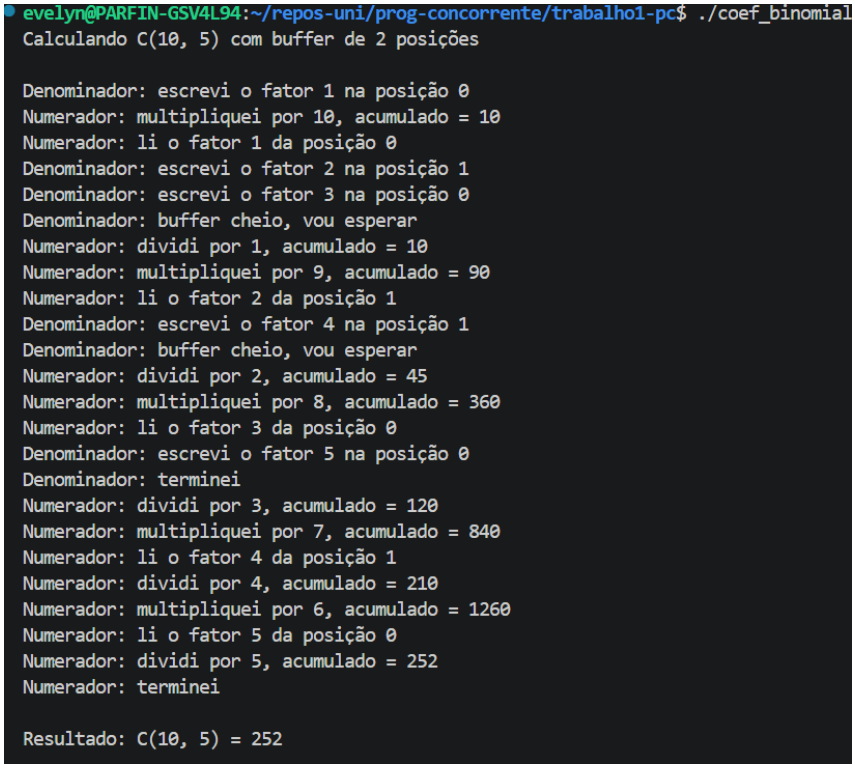


Figura 1: Saída da execução padrão ($n = 10$, $k = 5$, $N = 2$).

4 Conclusão

5 Referências

BEN-ARI, Mordechai. **Principles of Concurrent and Distributed Programming**. 2. ed. Harlow: Addison-Wesley, 2006. ISBN 978-0-321-31283-9.